

Ancient Ruin Excursion

You are participating in a conference hosted by the Amazing Pearl Innovative Organization (APIO) in Taiwan. The organization's mission is to showcase innovations related to bubble tea, which is a world-famous beverage made from tapioca pearls. However, you are not here to contribute your technical expertise; instead, you are here to enjoy the drinks and the excursions. During one such outing, you are brought to an ancient ruin. According to the guide, the site was only recently discovered, and its date of origin remains a mystery.

The guide explains that the ruin consists of N towers, labeled $0, 1, \dots, N - 1$, where the i -th tower has a height of $H[i]$. There are $N - 1$ bidirectional roads connecting these towers. Historical research indicates that tower 0 was the first to be constructed. For each subsequent tower i (where $i \geq 1$), builders selected a previously built tower $P[i]$ ($0 \leq P[i] < i$) to be its parent tower. For convenience, let $P[0] = -1$. Construction materials were stored in the parent tower. To facilitate transport, a bidirectional road was built specifically between $P[i]$ and i .

Unfortunately, many towers lie in partial ruin and the roads are buried under dust. Consequently, **all tower heights $H[i]$ and parent assignments $P[i]$ are unknown**. To uncover the secrets of this civilization, archaeologists have developed a *convenience detector*, a device used by the ancients to plan their city layouts.

The detector takes an input array $D[0], D[1], \dots, D[N - 1]$ and outputs an array $C[0], C[1], \dots, C[N - 1]$. It calculates the **convenience values** of the towers by assuming each tower i has a **modified height** $H'[i] = H[i] + D[i]$. For every tower i , its convenience value $C[i]$ is defined as the number of its adjacent towers j such that:

- $H'[j] < H'[i]$, or
- $j = P[i]$ and $H'[j] = H'[i]$.

A tower j is considered adjacent to tower i if there is a road connecting the two towers.

The host of APIO has announced a challenge. If you can determine all parent tower indices $P[i]$ and the **relative heights** $R[i] := H[i] - H[0]$, you will be awarded free bubble tea for the rest of the trip. Although absolute heights are impossible to determine, identifying these relative values will be enough to earn your prize.

Implementation details

You should implement the following procedure.

```
std::pair<std::vector<int>, std::vector<int>>
find_information(int N, int S)
```

- N : the number of towers in the ruin.
- S : the maximum possible height of the towers.
- This procedure is called **up to 1000 times** per test case.

The procedure should return a pair of two arrays, P' and R' , which represent the indices of the parent towers and the relative height differences, respectively, **excluding tower 0**.

- Both arrays must have a size of exactly $N - 1$.
- For each i such that $1 \leq i < N$, $P'[i - 1]$ must equal $P[i]$ and $R'[i - 1]$ must equal $R[i]$.

To interact with the detector, the procedure can call the following procedure:

```
std::vector<int>
detect_convenience_values(const std::vector<long long> &D)
```

- The argument D must be an array of exactly N elements. For every i ($0 \leq i < N$), $D[i]$ must satisfy $-10^{18} \leq D[i] \leq 10^{18}$.
- The procedure returns an array of N elements representing the detected convenience values.
 - This procedure can be called at most 700 times during each execution of `find_information`.

Constraints

- $2 \leq N \leq 10^5$
- The sum of N of all calls to `find_information` in a single test case does not exceed 10^5 .
- $1 \leq H[i] \leq S \leq 10^6$ for each i such that $0 \leq i < N$.
- $0 \leq P[i] < i$ for each i such that $1 \leq i < N$.

Subtasks and Scoring

Subtask	Score	Additional Constraints
1	2	$N = 2, S = 350$.
2	3	$N = 2, S = 10^6$.
3	6	$P[i] = 0, S = 10^6$.
4	9	$P[i] = i - 1, S = 10^6$.
5	4	$N \leq 700, S = 1$.
6	8	For each $1 \leq i < N$, there exists a path between tower 0 and i with at most two edges; $S = 1$.
7	13	$S = 1$.
8	55	$S = 10^6$.

In subtask 8, your score depends on the number of calls made to `detect_convenience_values`.

- If `detect_convenience_values` is called with invalid arguments, or if `find_information` returns an incorrect answer, your score for that subtask will be 0.
- Otherwise, let Q be the maximum number of calls made to `detect_convenience_values` during any single execution of `find_information` within the subtask. Your score will be calculated as follows:

Condition	Score
$700 < Q$	0
$38 < Q \leq 700$	$55 \times \max\left(1 + \log_{7.5}\left(\frac{18}{Q-20}\right), \frac{10}{55}\right)$
$Q \leq 38$	55

Example

Example 1

Consider the following call.

```
find_information(5, 1000000)
```

The hidden values are

- $P = [-1, 0, 1, 2, 3]$
- $H = [2, 6, 7, 5, 1]$

Suppose `find_information` calls:

```
detect_convenience_values([1, 0, -3, 0, 4])
```

The modified heights H' become $[3, 6, 4, 5, 5]$. The convenience values C are calculated as follows:

- $C[0] = 0$: Its only adjacent tower is tower 1. $H'[1] = 6$, which is not less than $H'[0] = 3$.
- $C[1] = 2$: Its adjacent towers are towers 0 and 2. $H'[0] = 3 < 6$ and $H'[2] = 4 < 6$. Both are strictly lower.
- $C[2] = 0$: Its adjacent towers are towers 1 and 3. $H'[1] = 6$ and $H'[3] = 5$; neither is less than or equal to $H'[2] = 4$.
- $C[3] = 1$: Its adjacent towers are towers 2 and 4. $H'[2] = 4 < 5$, so it is counted. For tower 4, $H'[4] = 5$. Although $H'[4] = H'[3]$, tower 4 is not the parent tower of tower 3, so it is not counted.
- $C[4] = 1$: Its only adjacent tower is its parent tower, tower 3. Since $H'[3] = 5$ and $H'[4] = 5$, the parent tower is counted into the convenience value.

Thus, `detect_convenience_values` returns $[0, 2, 0, 1, 1]$.

If `find_information` calls `detect_convenience_values([6, 0, 0, 0, 0])`, the modified heights are $[8, 6, 7, 5, 1]$, and the return value is $[1, 0, 2, 1, 0]$.

If `find_information` calls `detect_convenience_values([0, 1, 0, 0, 4])`, the modified heights are $[2, 7, 7, 5, 5]$, and the return value is $[0, 1, 2, 0, 1]$.

Finally, `find_information` should return:

```
([0, 1, 2, 3], [4, 5, 3, -1])
```

Note that the returned arrays should not contain $P[0]$ and $R[0]$.

Example 2

Consider the following call.

```
find_information(7, 1000000)
```

The hidden values are

- $P = [-1, 0, 0, 1, 1, 2, 2]$

- $H = [3, 1, 4, 1, 5, 9, 2]$

In this scenario, tower 0 is the parent of 1 and 2; tower 1 is the parent of 3 and 4; tower 2 is the parent of 5 and 6.

If `find_information` calls `detect_convenience_values([0, 0, 0, 0, 0, 0, 0])`, the modified heights are $[3, 1, 4, 1, 5, 9, 2]$, and the return value is $[1, 0, 2, 1, 1, 1, 0]$.

If `find_information` calls `detect_convenience_values([0, 2, -1, 2, -2, -6, 1])`, the modified heights are $[3, 3, 3, 3, 3, 3, 3]$, and the return value is $[0, 1, 1, 1, 1, 1, 1]$.

If `find_information` calls `detect_convenience_values([-2, 0, 0, 0, 0, 0, 0])`, the modified heights are $[1, 1, 4, 1, 5, 9, 2]$, and the return value is $[0, 1, 2, 1, 1, 1, 0]$.

Finally, `find_information` should return:

```
([0, 0, 1, 1, 2, 2], [-2, 1, -2, 2, 6, -1])
```

Sample Grader

The first line of the input should contain a single integer T , the number of scenarios. A description of T scenarios should follow, each in the format specified below.

Input format:

```
N S
P[1] P[2] ... P[N-1]
H[0] H[1] H[2] ... H[N-1]
```

Output format:

If your solution is correct, the grader outputs:

```
Accepted: Q
```

Otherwise, it outputs:

```
Wrong Answer: MSG
```

- Q : The number of calls made to `detect_convenience_values` in this scenarios.
- MSG : A message indicating the reason for the result, which may include:

- too many queries: `detect_convenience_values` was called more than 700 times in a single scenario.
- invalid call: `detect_convenience_values` was called with invalid arguments.
- incorrect answer: `find_information` returned an incorrect P' or R' array.

If multiple errors occur, the sample grader will only report one of them. The program will terminate immediately upon encountering its first error.

The implementation of the sample grader may differ from the actual grader.